

DOCUMENTO TÉCNICO

OPEN SOURCE

CONFIDENCIAL

Plataforma Web y Ecommerce Segura, Visual y Generativa con IA

Requisitos Técnicos, Arquitectura, Prompts Multiagente
y Planning hasta Producción

Basado en stack 100% Open Source · Seguridad por diseño · Desarrollo multiagente guiado
por humanos

Agosto 2026 · Documento de trabajo v1.0

Índice

1. Resumen ejecutivo
2. Requisitos técnicos (stack open source)
3. Arquitectura general
4. **PARTE CRÍTICA I** — DSL declarativo: la frontera de confianza
5. **PARTE CRÍTICA II** — Capa de capacidades y modelo de permisos
6. **PARTE CRÍTICA III** — Seguridad multi-tenant (PostgreSQL + RLS)
7. **PARTE CRÍTICA IV** — Sandbox de extensiones (WASM) y aislamiento
8. **PARTE CRÍTICA V** — Pruebas adversariales y puertas de calidad
9. Prompts de la fábrica multiagente
10. Planning completo hasta producción
11. Vertical slice recomendado (primer flujo real)
12. Riesgos, decisiones abiertas y KPIs

1. Resumen ejecutivo

El objetivo es construir una alternativa moderna a WordPress que combine la sencillez visual de Elementor con un núcleo seguro, modular, conectado a datos reales y asistido por IA. La plataforma se desarrolla mediante un sistema multiagente de IA guiado y aprobado por humanos, y se basa íntegramente en software open source.

Este documento se centra en **profundizar en las partes más sensibles y vitales del planning actual**: el DSL como frontera de confianza entre la IA y el runtime, el modelo de capacidades y permisos, el aislamiento multi-tenant con Row-Level Security, el sandbox de extensiones y las pruebas adversariales. Estas cinco áreas son las que determinan si la plataforma puede ir a producción de forma segura.

Principio rector no negociable: la IA **nunca** genera código libre ni SQL. Genera exclusivamente un DSL declarativo validado por esquema. Toda operación de datos pasa por la capa de capacidades. La seguridad surge de la arquitectura (restricciones, aislamiento, contratos, pruebas), no de la confianza en el LLM.

2. Requisitos técnicos (stack open source)

2.1 Stack base

Capa	Tecnología OSS	Licencia	Justificación
------	----------------	----------	---------------

Editor visual	React 18 + TypeScript + Vite	MIT	Ecosistema maduro drag&drop
Motor canvas / DnD	dnd-kit + Craft.js	MIT	Base editor tipo Elementor
Estado / historial	Zustand + Yjs (CRDT)	MIT	Undo/redo, colaboración
API núcleo	NestJS (Node) + Rust (Axum) en partes críticas	MIT / Apache-2.0	Contratos tipados, rendimiento
DSL / validación	JSON Schema + Zod/ajv	MIT	Declarativo, validable
Sandbox extensiones	WebAssembly (Wasmtime/WASI) + Workers	Apache-2.0	Aislamiento de recursos
Base de datos	PostgreSQL 16 + RLS	PostgreSQL	Multi-tenant nativo
Cache / colas	Redis + BullMQ	BSD / MIT	Jobs, rate limiting, sync
Media	MinIO (S3-compatible)	AGPLv3	Portabilidad de almacenamiento
Auth / MFA / RBAC	Keycloak o Ory (Kratos/Hydra)	Apache-2.0	OIDC, MFA, multi-tenant
Secretos	HashiCorp Vault / Infisical	MPL / MIT	Rotación, fuera del código
Render público	Astro / Next.js	MIT	SEO, SSG/SSR controlado
Observabilidad	OpenTelemetry + Grafana + Loki + Prometheus	Apache-2.0	Logs/auditoría sin secretos
CI/CD + GitOps	Gitea Actions / GitHub Actions + ArgoCD	MIT / Apache-2.0	Releases progresivos, rollback
Infra	Docker + Kubernetes (k3s) + Terraform + Unleash	Apache-2.0 / MPL	Aislamiento + feature flags
Orquestación agentes	LangGraph / CrewAI + LiteLLM	MIT	Multiagente con HITL
Memoria agentes	Qdrant / pgvector	Apache-2.0	Base conocimiento versionada

2.2 Requisitos funcionales clave

- **RF1:** Editor visual (canvas + panel de propiedades + responsive desktop/tablet/móvil + undo/redo/historial).
- **RF2:** Generación IA → salida SIEMPRE en DSL validado (nunca código libre).
- **RF3:** Árbol de componentes con props, bindings, actions, permisos y versión.
- **RF4:** Conectores de datos con contrato
`authenticate/importData/exportData/syncData/handleWebhook/validateConnection`.

- **RF5:** Publicación versionada con preview, rollback y feature flags.
- **RF6:** Migrador WordPress/WooCommerce (páginas, medios, productos, pedidos, SEO, redirecciones).
- **RF7:** Sistema de capacidades con permisos explícitos por componente.
- **RF8:** Marketplace de extensiones sandboxeadas (WASM) con manifiesto de permisos.

2.3 Requisitos no funcionales

- Multi-tenant con aislamiento a nivel de BD (RLS) + red + storage.
- LLM sin acceso directo a SQL; solo servicios de negocio autorizados.
- p95 < 200 ms API core; TTI editor < 3 s; páginas publicadas Lighthouse ≥ 90.
- Auditoría inmutable (append-only), backups cifrados, RPO 1 h / RTO 4 h.
- OWASP ASVS L2 mínimo; tests adversariales automatizados en CI.
- Licencias auditadas (FOSSA/ScanCode); núcleo AGPLv3 o Apache-2.0 (a decidir por modelo de negocio).

3. Arquitectura general



Fronteras de confianza (de menor a mayor privilegio): (1) el LLM produce DSL → (2) el validador de esquema lo acepta o rechaza → (3) el renderer solo pinta nodos validados → (4) las actions solo invocan capacidades declaradas → (5) las capacidades acceden a datos vía RLS. Cada frontera reduce lo que el nivel anterior puede hacer.

4. PARTE CRÍTICA I — DSL declarativo: la frontera de confianza VITAL

El DSL es el corazón de la seguridad y de la editabilidad. Es la única cosa que la IA puede producir y la única cosa que el runtime acepta renderizar. Si esta frontera es sólida, gran parte del riesgo desaparece.

4.1 Propiedades obligatorias del DSL

- **Declarativo:** describe qué, no cómo. Sin lógica ejecutable arbitraria.
- **Validable:** todo documento pasa por JSON Schema antes de renderizarse o guardarse.
- **Versionado:** cada componente y cada schema tienen versión semántica.
- **Portable:** permite sustituir un proveedor de datos por otro sin rediseñar la página.
- **Cerrado por defecto:** cualquier propiedad no declarada en el schema se rechaza (no se ignora).

4.2 JSON Schema (v1) del componente — versión de referencia

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://plataforma.dev/schemas/component-v1.json",
  "title": "ComponentNode",
  "type": "object",
  "additionalProperties": false,
  "required": ["type", "version"],
  "properties": {
    "type": { "type": "string",
      "enum": ["Page", "Header", "Hero", "Heading", "Text", "Button",
        "ProductGrid", "ProductCard", "Image", "Footer", "Section"] },
    "version": { "type": "string", "pattern": "^\\d+\\.\\.\\d+\\.\\.\\d+$" },
    "props": {
      "type": "object", "additionalProperties": false,
      "properties": {
        "theme": { "enum": ["light", "dark"] },
        "showRating": { "type": "boolean" },
        "text": { "type": "string", "maxLength": 5000 }
      }
    },
    "bindings": {
      "type": "object", "additionalProperties": false,
      "patternProperties": {
        "^[a-zA-Z][a-zA-Z0-9_]*$": {
          "type": "string",
          "pattern": "^(product|category|cart|user)\\.\\. [a-zA-Z0-9_\\.]+$"
        }
      }
    },
    "actions": {
      "type": "object", "additionalProperties": false,
      "patternProperties": {
        "^[a-zA-Z][a-zA-Z0-9_]*$": {
          "type": "string",
          "enum": ["cart.add", "cart.remove", "navigate", "form.submit"]
        }
      }
    }
  }
}
```

```

    }
  },
  "permissions": {
    "type": "array",
    "items": { "enum": ["catalog.read", "cart.read", "cart.write",
                        "media.read", "orders.read", "orders.update"] },
    "uniqueItems": true
  },
  "responsive": {
    "type": "object", "additionalProperties": false,
    "properties": {
      "desktop": { "type": "object" },
      "tablet": { "type": "object" },
      "mobile": { "type": "object" }
    }
  },
  "children": {
    "type": "array",
    "items": { "$ref": "#" },
    "maxItems": 200
  }
}

```

4.3 Ejemplo válido de componente generado por IA

```

{
  "type": "ProductCard",
  "version": "1.0.0",
  "props": { "theme": "dark", "showRating": true },
  "bindings": {
    "title": "product.name",
    "image": "product.image",
    "price": "product.price"
  },
  "actions": { "button": "cart.add" },
  "permissions": ["catalog.read", "cart.write"],
  "responsive": {
    "mobile": { "columns": 1 },
    "desktop": { "columns": 4 }
  }
}

```

Reglas de rechazo del validador (fail-closed):

- Cualquier `type` no listado en el enum → rechazo.
- Cualquier propiedad extra (`additionalProperties: false`) → rechazo, no se ignora.
- Bindings que no coinciden con el patrón de fuentes autorizadas → rechazo.
- Actions fuera del enum de operaciones de negocio → rechazo.
- Profundidad de árbol o número de nodos excesivos → rechazo (anti-DoS).
- Uso de una capacidad no declarada en `permissions` → rechazo en runtime.

4.4 Pipeline de validación (dónde se aplica)

Punto	Validación aplicada
Salida del LLM (AI Service)	Schema v1 + verificación de capacidades declaradas vs. disponibles para el tenant
Guardado (Core API)	Re-validación + sanitización de strings (anti-XSS) + límites de tamaño
Render (editor y público)	Solo se pintan nodos validados; strings escapados; bindings resueltos vía capacidades
Publicación (Publishing)	Validación final + snapshot versionado inmutable

5. PARTE CRÍTICA II — Capa de capacidades y permisos

VITAL

Los componentes nunca hablan con la base de datos ni con servicios externos directamente. Solo pueden invocar **capacidades**: operaciones de negocio con permisos mínimos, auditadas y limitadas por tenant.

5.1 Catálogo inicial de capacidades

Capacidad	Permite	Prohíbe explícitamente
<code>catalog.read</code>	Leer productos/categorías publicados del tenant	Leer productos de otro tenant o no publicados
<code>cart.read</code> / <code>cart.write</code>	Leer/añadir/quitar ítems del carrito del usuario	Modificar precios, aplicar descuentos no autorizados
<code>media.read</code>	Leer medios del tenant	Acceder a buckets de otros tenants
<code>orders.read</code>	Leer pedidos del usuario/rol autorizado	Leer pedidos ajenos
<code>orders.update</code>	Actualizar estado si el rol lo permite	Emitir reembolsos (requiere <code>orders.refund</code> + confirmación humana)

Ejemplo de aislamiento: un bloque `ProductGrid` con permisos `[catalog.read, cart.write]` puede leer productos publicados y añadir al carrito, pero NO puede: leer tokens/secretos, ejecutar SQL, consultar otro tenant, ni emitir reembolsos. Esto se prueba explícitamente en la suite adversarial.

5.2 Contrato de conectores de datos

```
interface DataConnector {
  authenticate(): Promise<AuthResult>;
  importData(query: TypedQuery): Promise<Dataset>;
  exportData(payload: TypedPayload): Promise<ExportResult>;
  syncData(cursor: SyncCursor): Promise<SyncResult>; // idempotente
  handleWebhook(req: SignedRequest): Promise<void>; // verifica firma
  validateConnection(): Promise<HealthStatus>;
}
```

Ejemplo de intercambiabilidad: `ProductGrid` → `CatalogProvider` → `WooCommerceAdapter`, y más adelante `ProductGrid` → `CatalogProvider` → `ShopifyAdapter`, sin rediseñar la página. El contrato incluye reintentos, idempotencia, logs y sincronización controlada.

6. PARTE CRÍTICA III — Seguridad multi-tenant (PostgreSQL + RLS) VITAL

El aislamiento entre tenants es la garantía más importante de un SaaS. Se implementa a nivel de base de datos con Row-Level Security, de modo que aunque una capa superior falle, la BD no devuelve datos de otro tenant.

6.1 DDL de referencia con RLS

```
-- Cada tabla de negocio lleva tenant_id
CREATE TABLE products (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL,
  name        TEXT NOT NULL,
  price       NUMERIC(12,2) NOT NULL,
  status      TEXT NOT NULL DEFAULT 'draft',
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- Activar RLS y forzarla incluso para el owner de la tabla
ALTER TABLE products ENABLE ROW LEVEL SECURITY;
ALTER TABLE products FORCE ROW LEVEL SECURITY;

-- El tenant se fija por conexión/transacción, nunca por el cliente
CREATE POLICY tenant_isolation ON products
  USING (tenant_id = current_setting('app.tenant_id')::uuid);

-- Solo lectura de productos publicados para el rol de render público
CREATE POLICY public_read_published ON products
  FOR SELECT
  USING (tenant_id = current_setting('app.tenant_id')::uuid
        AND status = 'published');
```

```
-- En cada transacción, el backend fija el tenant validado por Keycloak:
SET LOCAL app.tenant_id = '<uuid-del-tenant-autenticado>';
-- A partir de aquí, toda query solo ve filas de ese tenant.
```

Controles de seguridad obligatorios (del §13 del documento):

- El LLM no tiene acceso directo a SQL. Consultas parametrizadas y servicios de negocio controlados.
- Validación estricta de esquemas y entradas (Zod/ajv) en todas las capas.
- Permisos por usuario, organización, proyecto, recurso y tenant + RLS.
- MFA para administradores; cifrado y rotación de credenciales (Vault).
- Secretos fuera del código y jamás expuestos a componentes ni al frontend.
- Protección XSS, CSRF, inyección, abuso y rate attacks.
- Verificación de firmas e idempotencia en webhooks.
- Logs y auditoría sin contraseñas ni tokens; backups cifrados y rollback.
- Separación lectura/escritura; confirmación humana para acciones sensibles.

7. PARTE CRÍTICA IV — Sandbox de extensiones (WASM)

VITAL

Las extensiones de terceros son la mayor superficie de riesgo. En lugar de plugins con acceso total (el problema de WordPress), se ejecutan en un sandbox WebAssembly con capacidades explícitas y límites de recursos.

7.1 Manifiesto de extensión

```
{
  "name": "reviews-widget",
  "version": "0.3.1",
  "runtime": "wasm",
  "permissions": ["catalog.read"],
  "limits": {
    "cpu_ms": 50,
    "memory_mb": 32,
    "wall_time_ms": 200,
    "network": "none"
  },
  "signature": "<firma-del-publisher>"
}
```

7.2 Garantías de aislamiento

Dimensión	Control
CPU / memoria / tiempo	Límites duros en Wasmtime; se aborta al excederlos
Red	Denegada por defecto (WASI sin sockets); solo capacidades autorizadas vía host
Datos	Solo accede a capacidades declaradas en el manifiesto; el host media todo
Secretos	Nunca inyectados en el módulo WASM
Superficie	Núcleo pequeño con cambios restringidos; Rust/WASM en partes críticas

La arquitectura debe **probar explícitamente qué NO puede hacer** cada componente y extensión, no solo qué puede hacer.

8. PARTE CRÍTICA V — Pruebas adversariales y puertas de calidad VITAL

8.1 Roles de agentes enfrentados

- **Constructor:** genera componentes e integraciones.
- **Atacante:** intenta escapar del sandbox, saltar permisos, exfiltrar secretos, manipular precios, cruzar tenants.
- **Defensor:** propone restricciones, políticas y tests.
- **Auditor:** verifica el modelo de amenazas y puede bloquear un release.

8.2 Casos adversariales obligatorios (deben fallar / ser bloqueados)

Ataque	Resultado esperado
Ejecutar SQL desde el DSL	Rechazado por schema (no existe primitiva SQL)
Leer secretos / tokens	Imposible: no expuestos a componentes
Acceder a otro tenant	Bloqueado por RLS
Invocar <code>orders.refund</code> desde ProductGrid	Rechazado: capacidad no declarada
Manipular precios en el carrito	Precio se recalcula en servidor, no en cliente
Injectar XSS/CSRF	Sanitización + tokens CSRF + CSP
Agotar CPU/memoria en WASM	Abortado por límites del sandbox
Repetir webhooks	Idempotencia + verificación de firma
Escribir con permiso de solo lectura	Rechazado por capa de capacidades + RLS

8.3 Las 5 puertas de calidad (ningún release sin superarlas)

1. **Producto:** problema validado, usuario, métrica, alcance y exclusiones.
2. **Técnica:** contratos, API, migraciones, dependencias y rollback.
3. **Seguridad:** permisos, datos, amenazas, abuso y comportamiento no autorizado.
4. **Calidad:** tests, móvil, accesibilidad, edición posterior y reversión.
5. **Humana:** intención, claridad, experiencia y aprobación de publicación.

Releases progresivos: interno → alpha → beta privada → beta pública → rollout gradual → disponibilidad general, con feature flags, métricas de éxito/error, segmentos de usuarios, rollback y responsable humano.

9. Prompts de la fábrica multiagente

9.1 Política global (system prompt común)

POLÍTICA GLOBAL – Plataforma Web IA Segura (obligatoria):

1. Solo OSS con licencias compatibles (MIT, Apache-2.0, BSD, AGPL si aprobado). Registra la licencia de cada dependencia.
2. La IA genera EXCLUSIVAMENTE el DSL validado por JSON Schema. PROHIBIDO JavaScript arbitrario, SQL directo o código con acceso no declarado.
3. Todo acceso a datos pasa por la capa de capacidades. Declara permisos mínimos.
4. Seguridad por diseño: entradas validadas (Zod), consultas parametrizadas, RLS multi-tenant, secretos en Vault, logs sin credenciales.
5. Toda decisión relevante se registra como ADR en /docs/decisions.
6. Ningún merge ni release sin las 5 puertas: Producto, Técnica, Seguridad, Calidad, Humana.
7. Ante dudas o falta de info: PREGUNTA al humano; no asumas.
8. Salida estructurada, versionable y verificable (markdown + JSON si aplica).

9.2 Prompts por agente

Orquestador

Recibes un objetivo humano y lo descompones en tareas atómicas con: id, agente responsable, dependencias, entregable, criterio de aceptación y puerta asociada. Detecta conflictos, verifica cumplimiento de la Política Global y prepara para el humano un resumen con decisiones pendientes. No ejecutes tareas tú mismo. Salida: tabla de tareas + grafo de dependencias + decisiones humanas requeridas.

Producto

Convierte ideas y feedback en historias ("Como X quiero Y para Z"), criterios de aceptación Given/When/Then, prioridad RICE, alcance, exclusiones y preguntas abiertas. Cada historia referencia la métrica de éxito (ej: publicar landing en <30 min). No propongas solución técnica; define problema y resultado.

Investigación

Analiza plugins WordPress y competidores (Elementor, Webflow, Framer, Builder.io, Plasmic, Penpot). Registra por cada uno: problema, usuario, funcionalidad esencial, modelo de datos, integraciones, monetización, riesgo y reemplazo (núcleo/módulo/adaptador/extensión). Toda afirmación con fuente y nivel de confianza. Separa adopción, satisfacción y dependencia.

Arquitectura

Sobre el stack aprobado diseña: modelos de datos (DDL con RLS), APIs (OpenAPI 3.1), JSON Schema del DSL, manifiestos de permisos y contratos de conectores. Incluye alternativas, trade-offs y ADR. Prohíbe explícitamente qué NO puede hacer cada componente.

UX/UI

Diseña flujos del editor: canvas, panel de propiedades, drag&drop, edición inline, breakpoints, undo/redo, plantillas y asistente IA. Entrega user flows, wireframes, estados (vacío/carga/error/éxito), accesibilidad WCAG 2.1 AA y textos ES/EN. Prioriza simplicidad tipo Elementor.

Implementación

Desarrolla SOLO tareas aprobadas, sobre contratos de Arquitectura, en ramas aisladas. TypeScript estricto, validación Zod en toda entrada, tests unitarios en el mismo PR (cobertura $\geq 80\%$ en negocio), sin secretos hardcodeados, sin dependencias no aprobadas. El renderer del DSL rechaza cualquier nodo no validado.

Pruebas

Crea y ejecuta tests unitarios (Vitest), integración (Testcontainers+Postgres), E2E (Playwright), regresión visual, responsive (3 breakpoints) y accesibilidad (axe-core). Verifica los Given/When/Then de Producto. Bloquea la Puerta de Calidad si falta cobertura crítica.

Seguridad (Defensor/Auditor) — con poder de veto

Por cada cambio: actualiza modelo de amenazas (STRIDE), revisa permisos declarados vs usados, verifica RLS, analiza dependencias (osv-scanner, npm audit), valida firmas/idempotencia de webhooks y comprueba que nada expone secretos al LLM ni a componentes. Ejecuta checklist OWASP ASVS L2. Salida: informe con severidad (CVSS), bloqueos y remediaciones obligatorias.

Atacante (red team)

Tu único objetivo es romper la plataforma en entorno de pruebas: ejecutar SQL desde el DSL, leer secretos, cruzar tenants, invocar acciones no autorizadas, XSS/CSRF, manipular precios, agotar recursos WASM, repetir webhooks, escribir con permisos de lectura, exfiltrar datos no declarados. Documenta cada intento; cada éxito genera un ticket bloqueante para Seguridad.

Release

Prepara el candidato: changelog, migraciones con rollback, feature flags (Unleash), segmentación y checklist de las 5 puertas. Define camino progresivo interno-alpha-beta privada-beta pública-rollout gradual-GA, con métricas y responsable humano por fase. No publicas sin aprobación humana registrada.

Feedback/Soporte

Ingieres reviews, tickets, errores, telemetría y solicitudes. Clasifica por tipo, área, frecuencia, impacto, usuarios afectados, severidad y evidencia. Agrupa duplicados para detectar problemas reales. Propón priorización a Producto.

Generador de componentes (runtime IA)

Devuelve ÚNICAMENTE JSON que valide contra el JSON Schema del DSL v1. Estructura: type, version, props, bindings (datos vía capacidades), actions (operaciones autorizadas), permissions (mínimas), responsive. PROHIBIDO: código ejecutable, SQL, URLs de red arbitrarias, acceso a secretos, capacidades no declaradas. Si el usuario pide algo fuera de las capacidades, responde con error estructurado indicando qué capacidad falta.

10. Planning completo hasta producción

Equipo humano reducido + fábrica multiagente. Estimación orientativa: ~9–12 meses a GA.

Fase	Duración	Objetivo	Entregables clave	Puerta
F0 · Investigación	Sem 1–4	Validar problema y stack	Mapa de oportunidades y ADRs, matriz de plugins, benchmark	Producto
F1 · Fundaciones	Mes 2	Infra + seguridad base	K8s/Terraform, Keycloak, Vault, PostgreSQL+RLS, CI/CD, esqueleto multiagente	Técnica/ Seguridad
F2 · DSL + Runtime	Meses 3–4	Frontera de confianza	JSON Schema DSL v1, validador, renderer, capa de capacidades, sandbox WASM	Seguridad
F3 · Vertical Slice	Meses 4–5	Flujo real end-to-end	Flujo "ProductCard" completo (ver §11)	Las 5 puertas
F4 · Editor visual	Meses 5–7	Experiencia Elementor	Canvas, DnD, panel props, responsive, undo/redo, plantillas, asistente IA	Calidad
F5 · Datos e integraciones	Meses 7–8	Conectores	SDK integraciones, adaptador WooCommerce, mapeo visual, webhooks idempotentes	Seguridad
F6 · Migración WP/Woo	Mes 9	Sustitución progresiva	Migrador (páginas, medios, productos, pedidos, SEO, redirecciones) + informe	Técnica
F7 · Endurecimiento	Mes 10	Producción-ready	Pruebas adversariales completas, auditoría externa, backups/rollback, observabilidad, docs	Seguridad/ Humana
F8 · Releases progresivos	Meses 11–12	GA	interno → alpha → beta → rollout → GA con flags y métricas	Humana

10.1 Hitos de decisión (Go/No-Go humano)

- **Fin F0:** ¿construir propio o usar base existente? (según los 10 criterios del proyecto).
- **Fin F3:** ¿el vertical slice cumple métricas (publicar en <30 min, fidelidad, autorización sin fallos)?
- **Fin F7:** ¿auditoría externa superada? → habilita GA.

11. Vertical slice recomendado (primer flujo real)

"Generar una tarjeta de producto, conectarla a productos de prueba, verla en móvil y publicarla."

1. El usuario describe la tarjeta.
2. La IA genera el componente estructurado (DSL validado).
3. El usuario lo modifica visualmente.
4. Se conecta a un producto de prueba (vía `catalog.read`).
5. El botón ejecuta una acción autorizada de carrito (`cart.add`).
6. Se ejecutan pruebas visuales, responsive y de permisos.
7. Se publica una preview versionada.
8. Se recoge feedback.

Este slice ejercita simultáneamente las cinco partes críticas: DSL, capacidades, RLS, pruebas adversariales y puertas de calidad. Es el mejor validador temprano del riesgo técnico.

12. Riesgos, decisiones abiertas y KPIs

12.1 Decisiones abiertas a resolver en F0–F3

- Primer segmento de clientes: agencias, freelancers, ecommerce o empresas.
- Primera herramienta de diseño a integrar: Figma, Penpot o Canva.
- Nivel de fidelidad visual suficiente para la v1.
- Qué parte del runtime en Rust/WebAssembly.
- Qué capacidades entran en el núcleo tras el análisis de plugins.
- Modelo de monetización y licencia del núcleo (AGPL vs Apache).
- Proveedores externos iniciales y nivel de exportación.
- Auditorías externas necesarias antes de producción.

12.2 KPIs de éxito

KPI	Objetivo
Publicar landing/página de producto (usuario no técnico)	< 30 minutos
Tiempo instrucción/captura → componente editable	Minimizar (medir baseline)
Fidelidad visual vs. referencia	Alta, medible
Nº de correcciones manuales por componente	Reducir iterativamente
% componentes que conservan datos tras edición visual	> 95%

Errores de autorización detectados en pruebas	0 en release GA
Tiempo de migración desde WordPress/WooCommerce	Medible y decreciente
Tasa de rollback / errores / abandono	Baja, monitorizada

Idea central: "Diseña, describe o sube una referencia visual; la IA crea un componente editable, conectado a datos y limitado por permisos, mientras humanos y agentes colaboran para probar, publicar y mejorarlo."